

Lattices, LWE, and Regev's Cryptosystem

Lecture 7 of 15

SEC 595 — Encrypted Computing

- 1 Decoding With a Good and a Bad Basis
- 2 The Naïve Cryptosystem
- 3 Learning With Errors
- 4 Regev's Cryptosystem
- 5 Correctness and the Rounding Decision
- 6 A Worked Example: $q = 7$

Six lectures of foundations were built to reach this one.

Lattices, bases and the decoding problems came in Lecture 5. LWE, the error distribution and the noise bound came in Lecture 6.

Today all of it is spent at once, on the **first public-key encryption scheme of the course**: Regev, 2005.

Nothing new has to be built. The lecture can go straight to the cryptosystem.

Backward reference

Available without further work:

- a **lattice** $\mathcal{L}(B)$ and its **bases** — infinitely many for the same lattice;
- **good** and **bad** bases, measured by the orthogonality defect $\delta(B)$, and *Babai rounding* $B \lfloor B^{-1}t \rfloor$, which decodes correctly only when the basis is nearly orthogonal;
- **SVP**, **CVP** and **BDD** — CVP with a promise that the target is close, which makes the answer unique;
- the *q*-**ary lattices** $\Lambda_q(A)$ and $\Lambda_q^\perp(A)$, and the statement that an LWE sample is a BDD instance on $\Lambda_q(A)$.

Backward reference

Also available:

- **Search LWE** — recover s from samples $b_i = \langle a_i, s \rangle + e_i \bmod q$ — and **decision LWE**, distinguishing those samples from uniform; the two are equivalent.
- the **error distribution** $\mathcal{X}$, a discrete Gaussian, and the bound β with $\|e\|_\infty \leq \beta$ overwhelmingly often;
- **noise growth**: addition adds noise, multiplication multiplies it, and correctness is a comparison of the accumulated noise against a fraction of q .

Symbol	Meaning
q	the modulus; all arithmetic is in $\mathbb{Z}_q$
n	the length of the secret vector
m	the number of LWE samples: rows of A
A	a uniformly random matrix, $A \in \mathbb{Z}_q^{m \times n}$
s	the secret vector, $s \in \mathbb{Z}_q^n$
e	the error vector, small
b	the vector $As + e$; half of the public key
x	the encryption randomness, a binary vector
$\bar{b}$	the message: one bit
(c_1, c_2)	the ciphertext
Δ	the quantity decryption computes and then rounds

A collision of symbols, fixed now

The handout uses b for two different things:

- the **vector** $b = As + e$, published as part of the key;
- the **message bit**, written $\bar{b}$ with a bar.

In these slides the bar is always present on the message bit, and b without a bar is always the vector.

A third b appears in the opening basis example as a scalar coefficient. That one disappears once we reach Section 3.

Two bases, one exercise

$$B_{\text{bad}} = \left\{ \begin{pmatrix} 6 \\ 14 \end{pmatrix}, \begin{pmatrix} 3 \\ 8 \end{pmatrix} \right\} \qquad B_{\text{good}} = \left\{ \begin{pmatrix} 3 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \end{pmatrix} \right\}$$

The bad basis is two long, steep, nearly parallel vectors.

The good basis is two short orthogonal ones.

The task

A target point is given, off the lattice:

$$v = \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix}.$$

Write v as $a b_1 + b b_2$ with *real* coefficients, round a and b to integers, and see which lattice point comes back.

Do it once with each basis. That comparison is the whole point.

Backward reference

Lecture 5: given a target t and a basis B , *Babai rounding* returns $B \lfloor B^{-1}t \rfloor$ — write t in the basis, round each coefficient, multiply back.

The error it makes lies in the fundamental parallelepiped of B , so it is small exactly when the basis is nearly orthogonal. The guarantee is stated in terms of the smallest Gram–Schmidt norm.

Lecture 5 also recorded the consequence: *the public key is a bad basis, the secret key is a good one, and decryption is Babai decoding that only succeeds with the good basis.* This section is that sentence, worked.

Example — decoding with the bad basis

Solve

$$a \begin{pmatrix} 6 \\ 14 \end{pmatrix} + b \begin{pmatrix} 3 \\ 8 \end{pmatrix} = \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix}$$

for real a and b , round each to the nearest integer, and compute the lattice point that results.

The scan records the rounded pair twice, in two different colours, and the two disagree. Solve it yourself and decide which reading the arithmetic supports.

Example — decoding with the good basis

Now solve

$$a \begin{pmatrix} 3 \\ 0 \end{pmatrix} + b \begin{pmatrix} 0 \\ 2 \end{pmatrix} = \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix},$$

round, and compute the lattice point.

How much work was the second system compared with the first?

What the comparison shows

With the good basis the system is already diagonal: one division per coordinate, and the rounded answer is

$$4 \begin{pmatrix} 3 \\ 0 \end{pmatrix} + 2 \begin{pmatrix} 0 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 4 \end{pmatrix},$$

a lattice point close to v .

With the bad basis the coefficients are large — of order 10 and -20 — so rounding them moves the answer a long way. The recovered point is nowhere near v .

Same lattice. Same target. Same algorithm. Different basis.

Why large coefficients are fatal

Rounding changes each coefficient by at most $\frac{1}{2}$.

That half is then *multiplied by a basis vector*. With $\begin{pmatrix} 6 \\ 14 \end{pmatrix}$ and $\begin{pmatrix} 3 \\ 8 \end{pmatrix}$, half a unit of coefficient error is already several units of position error — and there are two such errors, which need not cancel.

With $\begin{pmatrix} 3 \\ 0 \end{pmatrix}$ and $\begin{pmatrix} 0 \\ 2 \end{pmatrix}$ the same half-unit costs at most 1.5 in x and 1 in y , in independent directions.

Keys

$$\text{PK} = B_{\text{bad}} = \left\{ \begin{pmatrix} 6 \\ 14 \end{pmatrix}, \begin{pmatrix} 3 \\ 8 \end{pmatrix} \right\}$$

$$\text{SK} = B_{\text{good}} = \left\{ \begin{pmatrix} 3 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \end{pmatrix} \right\}$$

Both bases describe the same lattice. The secret is not *which* lattice — it is a convenient description of it.

Enc

- 1 Choose integer coefficients (h, i) and form the lattice point $h b_1 + i b_2$ using the **public** basis.
- 2 Choose a small error vector $\vec{e}$.
- 3 The ciphertext is the perturbed point $h b_1 + i b_2 + \vec{e}$.

The handout's instance: $(h, i) = (14, -24)$, giving the lattice point $\begin{pmatrix} 12 \\ 4 \end{pmatrix}$,
and $\vec{e} = \begin{pmatrix} -0.4 \\ 0.2 \end{pmatrix}$, giving the ciphertext $\begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix}$.

The ciphertext is the target from Section 1

$$\begin{pmatrix} 12 \\ 4 \end{pmatrix} + \begin{pmatrix} -0.4 \\ 0.2 \end{pmatrix} = \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix}$$

The opening exercise was not a warm-up. It was a decryption attempt, run twice: once with the key an attacker has, once with the key the receiver has.

Dec

- 1 Write the ciphertext in the **secret** basis and round: this recovers the lattice point $\begin{pmatrix} 12 \\ 4 \end{pmatrix}$.
- 2 Re-express that lattice point in the **public** basis to recover the coefficients (h, i) .

Step 1 removes the error. Step 2 reads the message back out of the point.

Here step 2 returns $h = 14$, $i = -24$: exactly what was encrypted.

Example — the naïve scheme end to end

Using B_{bad} as PK and B_{good} as SK:

- a. encrypt the coefficient pair $(h, i) = (14, -24)$ with the error $\vec{e} = \begin{pmatrix} -0.4 \\ 0.2 \end{pmatrix}$;
- b. decrypt the result and recover (h, i) ;
- c. repeat step (b) using B_{bad} in place of B_{good} , and say what goes wrong.

What the trapdoor actually is

Everyone can *encrypt*: forming a lattice point needs only some basis, and adding a small error needs nothing at all.

Only the holder of the good basis can *decode* — undo the error and land back on the lattice point that was sent.

Encryption is easy in any basis; decoding is easy only in a good one.
That asymmetry is the whole construction.

Why this is only the naïve version

Everything so far happened in $\mathbb{R}^2$, with fractional coordinates and a fractional error.

Real cryptography needs

- exact arithmetic — so, $\mathbb{Z}_q$ rather than $\mathbb{R}$;
- a hardness assumption stated about *random* instances, not about two hand-picked bases.

Both are supplied by one object: **Learning With Errors**.

Learning With Errors

Given

- a random matrix A ,
- a secret vector s ,
- and an error vector e ,

all defined over $\mathbb{Z}_q$, form

$$As + e = b.$$

Three objects and one equation. Everything in the rest of the lecture is built out of that line.

The handout's margin note is the whole point:

$$Ax = b \quad \implies \quad x = A^{-1}b.$$

With $e = 0$, recovering the secret is Gaussian elimination over $\mathbb{Z}_q$ — fast, and completely standard.

Adding e destroys that. There is no A^{-1} trick that survives an unknown perturbation.

Backward reference

Lecture 5 gave the geometric reading of $b = A^T s + e$: the vector $A^T s$ is by definition a point of the q -ary lattice $\Lambda_q(A)$, so

b is a lattice point perturbed by the short vector e .

Recovering s means finding the nearest lattice point: CVP, and because e is short, BDD.

So Section 2's picture — a lattice point plus a small error, decoded with a trapdoor — and Section 3's equation are the same statement in two languages.

“Given A and b , is it easy to solve for s ?”

The answer is the security assumption of every scheme in the rest of this course: **no**, when e is drawn from $\mathcal{X}$ and A is uniform.

Lecture 6 recorded the evidence: search and decision LWE are equivalent, and Regev’s reduction ties the average case to worst-case lattice problems.

Example — with and without the error

Fix a small modulus q and a 3×3 matrix A over $\mathbb{Z}_q$ that you can invert by hand.

- a. Pick a secret s , publish $b = As$ with **no** error, and recover s .
- b. Now add an error e with entries in $\{-1, 0, 1\}$, publish $b = As + e$, and try the same computation. What does $A^{-1}b$ return?
- c. How many candidate secrets are consistent with your (A, b) ?

KeyGen

- **Secret key:** a random secret vector $s \in \mathbb{Z}_q^n$.
- **Public key:** a random matrix $A \in \mathbb{Z}_q^{m \times n}$, together with

$$b = As + e,$$

where e is a small error vector.

A is chosen freely; b is then determined by A , s and e .

$$\text{PK} = (A, b), \quad \text{SK} = s.$$

The public key is exactly m LWE samples, published together.

The error e is used once, at key generation, and then thrown away. Nobody ever needs it again — including the receiver.

$\text{Enc}(\text{PK}, \bar{b})$ — one bit

To encrypt a bit $\bar{b}$:

- 1 choose a random binary vector x ;
- 2 compute

$$\begin{aligned}c_1 &= A^T x \bmod q, \\c_2 &= b^T x + \bar{b} \lfloor q/2 \rfloor \bmod q.\end{aligned}$$

The ciphertext is (c_1, c_2) .

$$c_1 = \underbrace{A^T x}_{\text{a vector in } \mathbb{Z}_q^n}, \quad c_2 = \underbrace{b^T x}_{\text{a scalar}} + \underbrace{\bar{b} \lfloor q/2 \rfloor}_{\text{the message}}.$$

The random x takes a subset-sum of the published samples — itself a fresh LWE sample — and the message is then added to the scalar half.

c_1 carries no message at all. It exists so that the receiver can reconstruct, and subtract, the masking term inside c_2 .

Floor or ceiling?

The scan first writes

$$c_2 = b^T x + \bar{b} \lceil q/2 \rceil \bmod q,$$

strikes it out, and replaces it with

$$c_2 = b^T x + \bar{b} \lfloor q/2 \rfloor \bmod q.$$

The floor is the corrected version, and the one we use.

For even q the two agree. For odd q they differ by one, which matters only at the very edge of the decision rule below.

$\text{Dec}(s, (c_1, c_2))$

The receiver knows s , and computes

$$\Delta = c_2 - c_1^T s \bmod q.$$

The handout records what this should come out to:

$$\Delta \approx \bar{b} \lfloor q/2 \rfloor \bmod q.$$

One inner product and one subtraction. That is the entire decryption cost.

Rounding to a bit

$$\bar{b} = \begin{cases} 1 & \text{if } \Delta \text{ is closer to } q/2, \\ 0 & \text{otherwise.} \end{cases}$$

Decryption does not test Δ for equality with anything. It asks which of two landmarks — 0 and $q/2$ — the value is nearer to.

That tolerance is what lets the scheme survive an error term it never computes.

What Δ actually equals

Substitute the definitions. With $c_1 = A^T x$ and $b = As + e$:

$$c_1^T s = (A^T x)^T s = x^T As,$$

$$b^T x = (As + e)^T x = x^T As + e^T x.$$

Subtracting, the $x^T As$ terms cancel and

$$\Delta = \bar{b} \lfloor q/2 \rfloor + e^T x \bmod q$$

The message survives. What is left over is the error, weighted by the encryptor's random bits.

Why the cancellation happens at all

Neither party can compute $x^T A s$ on its own:

- the encryptor knows x and A but not s ;
- the receiver knows s and A but not x .

Each computes *one side* of the same quantity — the encryptor inside $b^T x$, the receiver inside $c_1^T s$ — and the subtraction destroys it.

This is the same shape as Diffie–Hellman: a shared value neither party sends.

Why the rounding decision works

Write the leftover as $\eta = e^T x$. Then

$$\Delta = \begin{cases} \eta & \text{if } \bar{b} = 0, \\ \lfloor q/2 \rfloor + \eta & \text{if } \bar{b} = 1. \end{cases}$$

So Δ sits near 0 in one case and near $q/2$ in the other, displaced by η in both.

Asking which landmark is nearer therefore returns $\bar{b}$ — provided η never pushes Δ more than a quarter of the way round the circle.

Correctness

Decryption returns the encrypted bit whenever

$$\left| e^T x \right| < \frac{q}{4} \pmod{q}.$$

Two things make $e^T x$ small: the entries of e are small by construction, and x is binary, so $e^T x$ is a sum of *some* of the entries of e and never a scaled-up multiple of them.

Two things make it large: a big m , and an error distribution with too much spread.

Backward reference

Lecture 6 replaced the distribution $\mathcal{X}$ by a bound β with $\|e\|_\infty \leq \beta$, precisely so that noise analysis becomes arithmetic.

Here that gives $|e^T x| \leq m\beta$ immediately, since x has at most m nonzero binary entries. The correctness condition becomes a parameter inequality:

$$m\beta < \frac{q}{4}.$$

“Choose q large enough relative to the noise” is the whole of parameter selection, in miniature.

Example — the correctness bound

Take $n = 4$, $m = 8$, and errors drawn uniformly from $\{-1, 0, 1\}$.

- a. What is the worst-case value of $|e^T x|$?
- b. What is the smallest q for which the bound of the previous slide is guaranteed?
- c. The worst case is very unlikely. Estimate the *typical* size of $e^T x$, and say what q that would suggest instead.

Forward reference

Regev encrypts **one bit**, and the lecture says nothing about computing on ciphertexts.

Lecture 8 writes down what a homomorphic scheme would have to provide, and Lecture 9 exhibits two schemes that provide half of it each.

The LWE machinery here returns in Lecture 12 as **GSW**, where the ciphertext becomes a matrix, and the noise term $e^T x$ becomes a quantity that must be tracked through every operation rather than bounded once.

The parameters

The handout works one full instance by hand, with

$$q = 7, \quad n = 3, \quad m = 3.$$

Nominally e should be real-valued in $[0, 1)$; the handout instead takes $e \in \mathbb{Z}_7^3$ so that the arithmetic stays integral.

Every step below is left blank. The scan carries struck-out values corrected in a second colour, and several of the corrections do not agree with each other — so we work it live rather than reprint it.

The given data

$$s = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad e = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}, \quad A = \begin{bmatrix} 2 & 1 & 3 \\ 4 & 0 & 6 \\ 5 & 2 & 1 \end{bmatrix}.$$

For the encryption:

$$\bar{b} = 1, \quad x = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

Example — key generation

Compute

$$b = As + e \bmod 7,$$

and write down the public key and the secret key.

With $\bar{b} = 1$ and $x = [1 \ 0 \ 1]^T$, compute

$$c_1 = A^T x \bmod 7, \quad c_2 = b^T x + \bar{b} \lfloor 7/2 \rfloor \bmod 7,$$

and state the ciphertext Alice sends.

The scan writes the summand $\lfloor 7/2 \rfloor$ as 4 on one line, records the corrected total as 3, and then sends 4 as the second component. At most one of those can be right — decide which, and say what the other two were.

Compute

$$c_1^T s \bmod 7 \quad \text{and then} \quad \Delta = c_2 - c_1^T s \bmod 7.$$

The scan writes $c_1^T s = 18$, which is the dot product *before* reduction. Reduce it, and check whether doing so changes Δ .

Example — does the decision rule return the right bit?

Apply the rule: is your Δ closer to 0 or to $q/2 = 3.5$, working modulo 7?

Compare your answer with the bit that was encrypted.

The scan's corrected reading is that $\bar{b} = 1$. Check it against your own computation, and against the correctness condition $|e^T x| < q/4$ for these parameters.

Example — repair the parameters

Keeping s , e , x and A as they are, find the smallest modulus q for which this instance decrypts correctly for *both* choices of $\bar{b}$.

Then explain, in one sentence, why $q = 7$ was never going to be enough.

- 1 A lattice has many bases. Decoding a perturbed lattice point by rounding coordinates works with a *good* basis and fails with a *bad* one, because rounding error is multiplied by the basis vectors.
- 2 That asymmetry is a trapdoor: publish the bad basis, keep the good one. Anyone can form a noisy lattice point; only the key holder can decode it.
- 3 Learning With Errors is the same statement in $\mathbb{Z}_q$: $As + e = b$ is a lattice point plus a short error, so recovering s is BDD. Delete e and it collapses to Gaussian elimination.

- 4 **Regev.** $SK = s$; $PK = (A, b)$ with $b = As + e$. Encrypt a bit as $c_1 = A^T x$, $c_2 = b^T x + \bar{b} \lfloor q/2 \rfloor$ for random binary x . Decrypt by $\Delta = c_2 - c_1^T s$.
- 5 Correctness is one cancellation: $\Delta = \bar{b} \lfloor q/2 \rfloor + e^T x$, because both parties compute $x^T As$ and it subtracts out.
- 6 The bit is recovered by asking whether Δ is nearer 0 or $q/2$ — correct whenever $|e^T x| < q/4$, i.e. whenever q is large enough relative to $m\beta$.
- 7 The message occupies the top of the modulus and the noise the bottom. That layout is the one Lecture 1 introduced as $\Delta m + e$.

Lecture 8 — Homomorphic Encryption: Definitions

Regev encrypts. It does not yet compute.

The next lecture writes down what a scheme must supply in order to compute on ciphertexts at all: a fourth algorithm, two correctness statements, and one commuting square.